UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 03

NOMBRE COMPLETO: Carballo Ramírez Hanny

Nº de Cuenta: 319236761

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 03

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 8 de marzo 2026

CALIFICACIÓN: _____

1. ACTIVIDADES REALIZADAS. UNA DESCRIPCIÓN DE LOS EJERCICIOS Y CAPTURAS DE PANTALLA DE BLOQUES DE CÓDIGO GENERADOS Y DE EJECUCIÓN DEL PROGRAMA

1.- Generar una pirámide rubik (pyraminx) de 9 pirámides por cara. Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña)

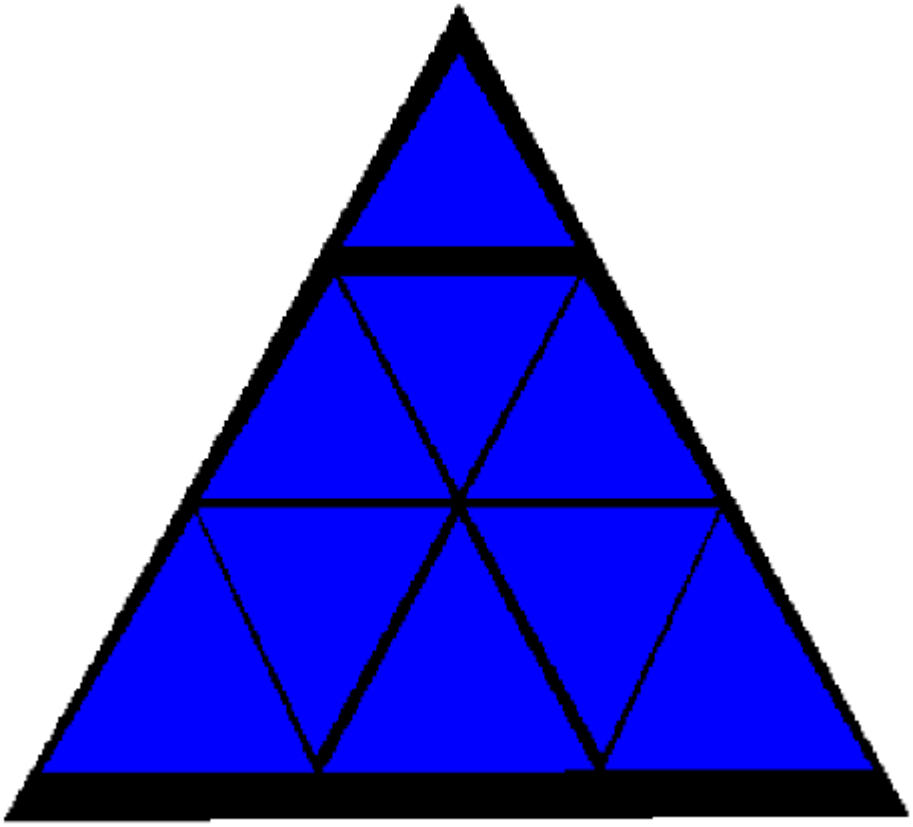
Agregar en su documento escrito las capturas de pantalla necesarias para que se vean las 4 caras de toda la pyraminx o un video en el cual muestra las 4 caras

Para llevar a cabo esta práctica se emplearon pirámides triangulares como primitivas geométricas, así como ecuaciones de transformación, las cuales permiten posicionar y organizar cada una de las pirámides dentro de la estructura completa.

```
// PRÁCTICA 03
//
// PIRÁMIDE BASE COLOR NEGRO
model = glm::mat4(1.0);
// Movimiento inicial para colocar el objeto hacia el eje -Z
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
// aquí se pueden aplicar otras transformaciones al objeto
//model = glm::scale(model, glm::vec3(0.0f,0.0f,0.0f));
//model = glm::scale(model, glm::vec3(1.5f,1.5f,1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
// la proyección normalmente se envía una sola vez,
// excepto si se cambia entre proyección ortogonal y perspectiva en ejecución
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //se asigna el color del objeto
meshList[1]->RenderMesh(); //renderiza el cubo o la pirámide triangular
//meshList[3]->RenderMeshGeometry(); //renderiza cilindro, cono o pirámide cuadrangular
//sp.render(); //renderiza una esfera

// PIRÁMIDE AZUL - CARA 1 / PIEZA 1
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// se repite esta traslación para ubicar las figuras en la misma zona del eje -Z
model = glm::translate(model, glm::vec3(0.0f, 0.3f, -2.34f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE indica que la matriz no se transpone
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //se establece el color del objeto
meshList[1]->RenderMesh(); //se dibuja la pirámide

// PIRÁMIDE AZUL - CARA 1 / PIEZA 2
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// traslación para posicionar la figura en la escena
model = glm::translate(model, glm::vec3(0.0f, 0.05f, -2.2f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 65 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(-0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh(); //dibujo de la figura

// PIRÁMIDE AZUL - CARA 1 / PIEZA 3
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// ajuste de posición dentro de la estructura
```

```
color = glm::vec3(0.0f, 0.0f, 1.0f);
// ajuste de posición dentro de la estructura
model = glm::translate(model, glm::vec3(0.0f, -0.3f, -2.02f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh(); //render de la pirámide

// PIRÁMIDE AZUL - CARA 1 / PIEZA 4
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// movimiento lateral para acomodar la pieza
model = glm::translate(model, glm::vec3(-0.31f, -0.3f, -2.02f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 8 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh(); //se dibuja el objeto

// PIRÁMIDE AZUL - CARA 1 / PIEZA 5
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// desplazamiento hacia el lado contrario
model = glm::translate(model, glm::vec3(0.31f, -0.3f, -2.02f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 8 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh();

// PIRÁMIDE AZUL - CARA 1 / PIEZA 6
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// ajuste de posición en el centro de la cara
model = glm::translate(model, glm::vec3(0.1575f, -0.01f, -2.18f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh();

// PIRÁMIDE AZUL - CARA 1 / PIEZA 7
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// posición simétrica respecto a la anterior
model = glm::translate(model, glm::vec3(-0.1575f, -0.01f, -2.18f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh();

// PIRÁMIDE AZUL - CARA 1 / PIEZA 8
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// colocación en la parte inferior izquierda
model = glm::translate(model, glm::vec3(-0.1575f, -0.23f, -2.04f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 65 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(-0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh();

// PIRÁMIDE AZUL - CARA 1 / PIEZA 9
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 1.0f);
// colocación en la parte inferior derecha
model = glm::translate(model, glm::vec3(0.1575f, -0.23f, -2.04f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
model = glm::rotate(model, 65 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(-0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh();
```

```
        // ------------------------------------------------------------

        // PIRÁMIDE VERDE - CARA 2 / PIEZA 1
        model = glm::mat4(1.0f);
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        // posicionamiento inicial de esta cara
        model = glm::translate(model, glm::vec3(0.15f, 0.3f, -2.5f));
        model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.2f));
        model = glm::rotate(model, 10 * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
        model = glm::rotate(model, 15 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
        model = glm::translate(model, glm::vec3(-0.5f, 0.0f, -0.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[1]->RenderMesh(); //render de la pirámide

        glUseProgram(0);
        mainWindow.swapBuffers();
    }
    return 0;
}
```



2. PROBLEMAS PRESENTADOS. LISTAR SI SURGIERON PROBLEMAS A LA HORA DE EJECUTAR EL CÓDIGO

En esta ocasión no fue posible finalizar la práctica como se esperaba. Aunque se logró aplicar correctamente las funciones de transformación para algunas partes del modelo, surgieron dificultades al intentar generar las demás caras del Pyraminx. Cada vez que se aplicaban traslaciones o rotaciones, las pirámides pequeñas no se visualizaban de manera correcta dentro de la escena. Debido a estos problemas en la manipulación de las transformaciones y en la correcta visualización de las figuras, se decidió dejar la práctica incompleta.

3. CONCLUSIÓN:

A. LOS EJERCICIOS DE LA CLASE: COMPLEJIDAD, EXPLICACIÓN

B. COMENTARIOS GENERALES: FALTÓ EXPLICAR A DETALLE, IR MÁS LENTO EN ALGUNA EXPLICACIÓN, OTROS COMENTARIOS Y SUGERENCIAS.

A. Los ejercicios de la clase:

Los ejercicios realizados en esta práctica presentaron un nivel de complejidad considerable, ya que fue necesario aplicar transformaciones geométricas como traslaciones y rotaciones para posicionar correctamente las pirámides pequeñas y formar la estructura del Pyraminx. Aunque se comprendió el uso de estas transformaciones, resultó complicado acomodar cada instancia de manera adecuada para que todas las caras se visualizaran correctamente.

B. Comentarios generales:

Durante la realización de la práctica surgieron algunas dudas relacionadas con la aplicación de las transformaciones y la forma correcta de organizar las figuras dentro de la escena. Considero que sería útil dedicar un poco más de tiempo a explicar paso a paso cómo aplicar estas transformaciones en estructuras más complejas. A pesar de las dificultades, la práctica permitió comprender mejor cómo se manipulan objetos en un entorno de gráficos 3D.